FORGEOPS ENGINEERING HANDBOOK

The Complete Cloud, DevOps & Site Reliability Engineering Master Handbook

Author: Staff Platform & DevOps Architect
Scope: Microsoft Azure, Docker, Kubernetes, Helm 3, Terraform, Prometheus & Grafana
Document Version: 1.0.0 (Production Architecture Edition)

Chapter 1: Microsoft Azure & Enterprise Cloud Architecture

1.1 Cloud Service Models

- **IaaS (Infrastructure as a Service)**: You rent raw compute, virtual machines (Azure VMs), storage, and virtual networks. You manage the OS, middleware, patching, data, and applications. Azure manages virtualization, servers, storage, and networking.
- **PaaS (Platform as a Service)**: The cloud provider manages the operating system, patching, runtime, and hardware. You only manage application code and data. Examples: Azure App Service, Azure Container Apps, Azure SQL.
- **SaaS (Software as a Service)**: Fully managed software accessible over the web. Examples: Microsoft 365, Dynamics 365.

1.2 Azure Resource Hierarchy & Governance

Azure organizes every asset in a strict 4-level management hierarchy:

```
[1] Microsoft Entra ID Tenant (Identity & Security Boundary)
      └── [2] Management Groups (Governance & Azure Policy Boundary)
            └── [3] Azure Subscription (Billing & Quota Boundary)
                  └── [4] Resource Groups (Lifecycle & Containment Boundary)
                        ├── Virtual Network (VNet)
                        ├── Azure Kubernetes Service (AKS)
                        ├── Azure Container Registry (ACR)
                        └── Azure Key Vault
```

Core Principle: If you delete a **Resource Group**, Azure terminates and deletes every resource inside it. This is why Terraform groups environments into dedicated Resource Groups.

1.3 Azure Global Infrastructure

- **Regions**: Geographic areas containing multiple physical data centers connected through a dedicated low-latency network (e.g., `East US`, `West Europe`).

- **Availability Zones (AZs)**: Unique, physically separate data centers within an Azure Region. Each zone has independent power, cooling, and networking. Running workloads across 3 AZs provides a **99.99% uptime SLA** against data center fires or electrical failures.
- **Paired Regions**: Each Azure region is paired with another region in the same geography (e.g., `East US` paired with `West US`) for sequential platform updates and disaster recovery.

1.4 Azure Networking Deep Dive

- **Virtual Network (VNet)**: A private, isolated network in Azure with customizable IP address blocks (e.g., `10.0.0.0/16` = 65,536 private IPs).
- **Subnets**: Subdivisions of a VNet (e.g., `10.0.1.0/24` = 256 IPs) isolating tiers (frontend subnets, AKS node subnets, database subnets).
- **Network Security Groups (NSGs)**: Stateful firewall filters controlling inbound and outbound traffic using 5-tuple evaluation (Source IP, Source Port, Destination IP, Destination Port, Protocol).
- **VNet Peering**: Directly connects two Azure VNets using Microsoft's private optical backbone with zero internet exposure and ultra-low latency.
- **Azure Bastion**: A fully managed PaaS service providing secure RDP/SSH access to VMs directly through the Azure Portal via TLS (port 443), eliminating public IP addresses on servers.

1.5 Azure Container & Identity Services

- **Azure Container Registry (ACR)**: Private OCI-compliant registry. Hardening best practice: set `admin_enabled = false` and authenticate nodes via Managed Identities rather than permanent root passwords.
- **Azure Kubernetes Service (AKS)**: Managed Kubernetes. Microsoft manages the control plane (`kube-apiserver`, `etcd`) for free; you only pay for worker node VMs.
- **Managed Identities**: Azure injects tokens via a local Instance Metadata Service (IMDS) at `http://169.254.169.254`. The AKS Kubelet identity is bound to ACR using the **`AcrPull`** RBAC role. No credentials ever exist in config files or secrets.

Chapter 2: Docker & Containerization Deep Dive

2.1 How Containers Work Under the Hood

Containers are **NOT** virtual machines. A virtual machine virtualizes hardware and runs a full guest operating system. A container is an **isolated Linux process** running directly on the host kernel.

Linux Mechanism	Category	Responsibility in Containers
`PID Namespace`	Namespaces (Visibility)	Process isolation (inside the container, your app is PID 1).
`NET Namespace`	Namespaces (Visibility)	Network virtualization (each container gets its own virtual `eth0` and IP).
`MNT Namespace`	Namespaces (Visibility)	Filesystem isolation (restricts process to container rootfs).
`IPC Namespace`	Namespaces (Visibility)	Inter-Process Communication isolation.
`cgroups (Control Groups)`	Resource Bounds	Restricts CPU quotas, memory limits, and disk I/O. Triggers **OOM Killer** if memory limit is exceeded.
`OverlayFS (Overlay2)`	Storage Driver	Stacks multiple read-only image layers together with a thin read-write container layer on top using copy-on-write (CoW).

2.2 Production Dockerfile Standards

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONPATH=/app

WORKDIR /app

# Layer caching: copy dependencies first
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app
COPY tests ./tests

# Security: non-root user execution
RUN useradd --create-home --uid 1000 appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000

# JSON exec form ensures proper POSIX SIGTERM handling for graceful shutdown
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

> **Critical Difference: Exec Form vs Shell Form**
> Always use `CMD ["node", "app.js"]` (exec form) rather than `CMD node app.js` (shell form). The shell form spawns `/bin/sh -c`, which swallows `SIGTERM` signals sent by Kubernetes, causing rolling updates to stall and forcefully kill pods.

Chapter 3: Kubernetes (K8s) & Helm Orchestration Mastery

3.1 The Kubernetes Architecture

- **Control Plane**:
  - `kube-apiserver`: The front door of the cluster. All internal and external requests (`kubectl`) authenticate and validate through this REST API.
  - `etcd`: A highly available, distributed, key-value database implementing the Raft consensus algorithm. Stores the entire desired and actual cluster state.
  - `kube-scheduler`: Watches for newly created unscheduled Pods. Evaluates node resource availability, affinity, and taints/tolerations to assign the pod to the best worker node.
  - `kube-controller-manager`: Runs continuous reconciliation loops. Compares current state with desired state (e.g., maintaining replica counts).
- **Worker Nodes**:
  - `kubelet`: The node agent. Watches the API server for PodSpecs assigned to its node and instructs the container runtime (`containerd`) to start or stop containers.
  - `kube-proxy`: Manages network routing rules on worker nodes (via Linux `iptables` or IPVS), load-balancing traffic across pods backing a `Service`.
  - `containerd`: Pulls images, manages storage layers, and starts containers using Linux namespaces and cgroups.

3.2 Core Workload & Networking Objects

- **Pod**: The atomic unit of deployment. Contains one or more tightly coupled containers sharing localhost (`127.0.0.1`), storage volumes, and network namespace.
- **Deployment**: Higher-level abstraction over ReplicaSets. Manages declarative zero-downtime rolling updates and rollbacks.
  - `maxSurge: 1`: Allows 1 extra pod above desired count during updates.
  - `maxUnavailable: 0`: Guarantees 0 pods are terminated until new pods pass their readiness checks.
- **Service Types**:
  - `ClusterIP` (Default): Exposes service internally within the cluster network only.
  - `NodePort`: Exposes service on a static high port (`30000-32767`) on every physical node IP.
  - `LoadBalancer`: Integrates with cloud providers (Azure Load Balancer) to provision an external public IP routing into the cluster.
  - `Headless Service` (`clusterIP: None`): Returns direct Pod IPs via DNS rather than a virtual IP (used for stateful clusters like Kafka, Redis).
- **Probes**:

- **Liveness Probe**: Detects deadlocks or unrecoverable application crashes. If it fails, Kubernetes restarts the container.
- **Readiness Probe**: Determines if the application is ready to accept user traffic. If it fails, Kubernetes removes the pod from the Service endpoint pool.
- **Startup Probe**: Disables liveness checks during slow legacy application initialization to prevent premature restart loops.

3.3 Helm 3 Mastery

Helm acts as both package manager and Go-templating engine for Kubernetes manifests:

```
helm lint charts/forgeops-api              # Validate chart syntax
helm template charts/forgeops-api          # Render templates to stdout
helm upgrade --install forgeops-api ./charts/forgeops-api \
  --namespace forgeops --create-namespace \
  --set image.tag=v1.0.0 --atomic --timeout 5m
```

Chapter 4: Terraform & Infrastructure as Code (IaC) Mastery

4.1 Principles of Infrastructure as Code

- **Declarative**: Define *what* the infrastructure should be, not *how* to build it.
- **Idempotency**: Running `terraform apply` 100 times produces the exact same end state as running it once.
- **Immutable Infrastructure**: Instead of modifying servers in place, replace them with newly versioned resources.

4.2 State Management & Remote State Locking

`terraform.tfstate` is a JSON ledger mapping declared resources to actual cloud IDs. In enterprise architectures, state is stored in **Azure Blob Storage** with lease locking:

```
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-forgeops-tfstate"
    storage_account_name = "stforgeopstfstate"
    container_name       = "tfstate"
    key                  = "forgeops.dev.terraform.tfstate"
  }
}
```

4.3 The 4 Core Terraform Commands

Command	Purpose
`terraform init`	Initializes working directory, downloads AzureRM provider plugins, and connects to remote backend.
`terraform plan`	Calculates the Directed Acyclic Graph (DAG) diff between real cloud state and desired code (+ add, ~ change, - destroy).
`terraform apply`	Executes Azure REST API calls to match declared configuration.
`terraform destroy`	Deletes all resources managed by the state file in reverse dependency order.

Chapter 5: Prometheus & Grafana Observability (SRE Mastery)

5.1 Google SRE's 4 Golden Signals

1. **Latency**: Time taken to service a request (tracked via p50, p90, and p99 percentiles).
2. **Traffic**: Demand on the system (HTTP Requests Per Second).
3. **Errors**: Rate of failed requests (HTTP 5xx responses).
4. **Saturation**: How full your system's computing capacity is (Memory %, CPU %, Disk I/O).

5.2 The 4 Prometheus Metric Types

Metric Type	Behavior	Production Example
Counter	Monotonically increasing integer (only increments or resets to 0).	`api_requests_total{method="GET", status="200"}`
Gauge	Numerical value that can arbitrarily go up or down.	`active_users_current`, `memory_bytes_used`
Histogram	Samples observations into configurable buckets and counts totals.	`api_request_duration_seconds_bucket{le="0.1"}`
Summary	Calculates configurable quantiles client-side over sliding time windows.	Request payload sizes.

5.3 Essential PromQL Formulas

```
# 1. Total Requests Per Second (RPS)
sum(rate(api_requests_total[1m]))

# 2. HTTP 5xx Error Rate Percentage
(sum(rate(api_requests_total{status=~"5.."}[1m])) / sum(rate(api_requests_total[1m]))) * 10

# 3. 95th Percentile Response Time (Latency)
histogram_quantile(0.95, sum by (le) (rate(api_request_duration_seconds_bucket[1m])))

# 4. Target Availability Alert Rule
up{job="forgeops-api"} == 0
```

Chapter 6: Commonly Used CLI Cheat Sheet

Docker CLI

```
docker run -d --name api -p 8000:8000 --restart unless-stopped forgeops-api:latest
docker ps -a                                # List all running and stopped containers
docker stop <container_id>                  # Send SIGTERM, then SIGKILL after 10s
docker kill <container_id>                  # Send immediate SIGKILL (crash simulation)
docker rm -f <container_id>                 # Force remove container
docker logs -f --tail 100 api               # Follow live log stream
docker exec -it api /bin/sh                 # Interactive shell inside container
docker inspect api                          # Detailed JSON metadata (IPs, mounts)
docker build -t forgeops-api:1.0.0 .        # Build container image
docker system prune -af --volumes           # Clean dangling containers & images
```

Kubernetes (kubectl) CLI

```
kubectl cluster-info                        # Display API server endpoint
kubectl get nodes -o wide                   # List physical/virtual worker nodes
kubectl get pods -n forgeops                # List pods in namespace
kubectl describe pod <pod_name> -n forgeops # View detailed events and failure reasons
kubectl logs -f <pod_name> -n forgeops      # Stream live pod logs
kubectl delete pod <pod_name> -n forgeops   # Terminate pod (tests self-healing)
kubectl apply -k k8s/                       # Apply kustomize resources
kubectl rollout status deployment/api -n ns # Monitor zero-downtime rollout
kubectl rollout undo deployment/api -n ns   # Rollback to previous revision
kubectl scale deployment/api --replicas=5   # Manually scale pod replicas
kubectl port-forward svc/forgeops-api 8080:80 -n forgeops
```

Helm 3 CLI

```
helm lint charts/forgeops-api               # Validate chart syntax
helm template charts/forgeops-api           # Render templates to stdout
helm upgrade --install forgeops-api ./charts/forgeops-api -n forgeops --atomic --timeout 5m
helm list -n forgeops                       # List active releases
helm history forgeops-api -n forgeops       # View release revision history
helm rollback forgeops-api 2 -n forgeops    # Instant rollback to revision 2
```

Terraform CLI

```
terraform -chdir=terraform init            # Initialize backend & download providers
terraform -chdir=terraform fmt             # Auto-format HCL code to standard
terraform -chdir=terraform validate        # Check code syntax and consistency
terraform -chdir=terraform plan            # Calculate diff and preview actions
terraform -chdir=terraform apply           # Provision live cloud infrastructure
terraform -chdir=terraform destroy         # Tear down all managed cloud resources
terraform -chdir=terraform state list      # List all resources tracked in state
```

Azure CLI (az)

```
az login                                   # Interactive browser login to Azure
az account show                            # View active subscription and tenant details
az account set --subscription <sub_id>     # Switch active billing subscription
az group list -o table                     # List all Resource Groups
az aks list -o table                       # List all AKS clusters
az aks get-credentials -g rg-name -n aks-name --overwrite-existing # Sync kubeconfig
az acr login --name acrname                # Authenticate local Docker with ACR
```